

Summary of "HybridFlow: September 30 2025"

Kai Jun Han (kaijun), Gary Huang (ioi), Tongyuan Miao (tymiao), Annie Jiang (annij)

Problem and Motivation

This work addresses the inefficiency and inflexibility of existing frameworks for Reinforcement Learning with Human Feedback (RLHF). The RLHF dataflow involves multiple large models with different computational profiles which creates significant performance bottlenecks that existing systems fail to solve adequately.

A primary challenge is the conflict between the two dominant distributed computing paradigms. Frameworks using a single-controller model offer flexibility in the dataflow, however it becomes a performance bottleneck due to high operator dispatch overhead. Conversely, the multi-controller paradigm is efficient for computation but suffers from inflexibility due to the design tightly coupling computation with data transfer, making the implementation of new algorithms or optimizations difficult.

Execution Bottlenecks

The actor model is a major source of inefficiency within the RLHF pipeline. It performs two distinct and demanding tasks: memory-bound token generation and computation-bound training. Using a single parallelism strategy for both is suboptimal, but switching between different, more optimal strategies creates significant communication and memory overhead in current systems.

Suboptimal Resource Use

Existing frameworks rely on rigid, hard-coded strategies for placing the various RLHF models on available GPUs. These static plans cannot adapt to different model sizes or cluster scales. As a result, they often lead to much GPU idle time and underutilization.

Related Works

This paper positions HybridFlow as a second generation solution to the RLHF systems problem. It argues that prior frameworks were foundational, first-generation systems focused on making RLHF feasible at scale, often at the cost of flexibility and fine-grained efficiency.

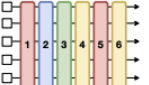
First-Generation Systems

The prior state-of-the-art was defined by frameworks like DeepSpeed-Chat, OpenRLHF, and NeMo-Aligner. These were creating systems that enabled the PPO algorithm to run on massive LLMs. Their main contribution was adapting distributed training libraries for the multi-model RLHF workflow. However, they were largely monolithic engineering solutions, hard-coded for a specific PPO dataflow with simplistic assumptions about resource management.

Limitations of Prior Work

The first-generation systems suffered from several limitations. Their rigid, non-modular architectures made it difficult for adaptation beyond PPO. Their solutions for handling the actor model's dual role were inefficient compromises, such as using two redundant model copies or forcing suboptimal parallelism for one of the stages. Finally, their use of static resource allocation with fixed placement strategies meant they could not adapt to varying workloads and hardware layout.

Table 1. Comparison of RLHF frameworks. Figures illustrate execution of one PPO iteration. Numbers 1-6 represent response generation, reward model inference, reference model inference, critic inference, actor training, and critic training, respectively.

RLHF system	DeepSpeed-Chat	OpenRLHF	NeMo-Aligner	HybridFlow
Parallelism	Training: ZeRO Generation: TP	Training: ZeRO Generation: TP	3D Parallelism for both training and generation	Training: 3D, ZeRO, FSDP Generation: 3D Parallelism
Actor weights in training & generation	Model resharding from ZeRO to TP	Using two copies of actor weights for the two stages	Using identical model partition in two stages (shared weights)	Zero-redundancy model resharding
Model Placement	Colocate all models on the same set of devices	Each model placed on separate devices	Actor/Ref colocated on some GPUs Critic/RM colocated on other GPUs	Support various model placement
Execution Pattern Actor $\square$ GPU Process Critic $\square$ Reward model Reference Policy $\square$				Support various execution patterns

Proposed Solution

HybridFlow's solution is a flexible and efficient framework built on a novel hierarchical hybrid programming model. This model strategically combines the strengths of two different distributed computing paradigms to manage the complex RLHF dataflow with both high performance and programmability.

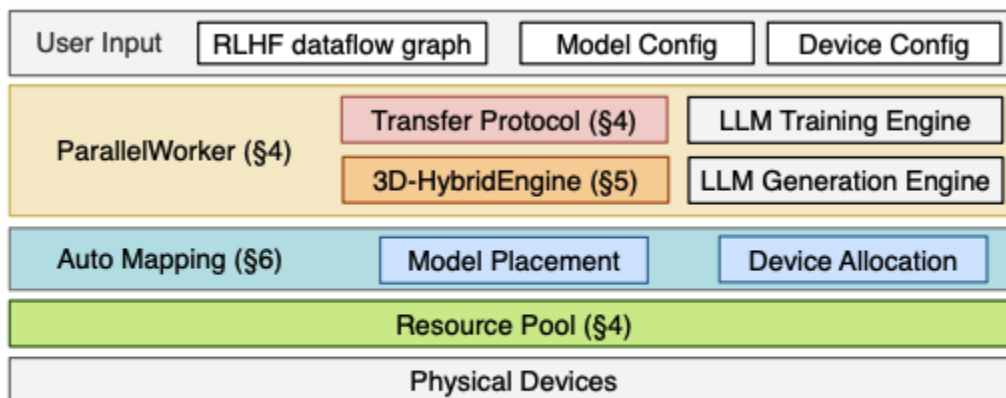


Figure 4. Architecture of HybridFlow.

The Hybrid Model

At the high level (inter-node), a single controller flexibly orchestrates the dataflow and dependencies between the different models in the RLHF graph. At the low level (intra-node), the efficient multi-controller paradigm manages the distributed computation within each individual

model. This design decouples the overall algorithm logic from the complex underlying computation, enabling both flexibility and efficiency.

Component 1: Hierarchical APIs

The framework provides a set of APIs that dramatically simplify algorithm development. These APIs encapsulate complex distributed computations into high-level model classes and use Transfer Protocols to abstract away the intricate details of data resharding between models. This allows developers to implement new RLHF algorithms in just a few lines of code.

Component 2: 3D-HybridEngine

This is a specialized engine designed to efficiently manage the actor model's training and generation stages on the same set of devices. Through a novel parallel grouping method, it achieves zero memory redundancy and reduces communication overhead during the model resharding step between stages.

Component 3: Auto Device Mapping

HybridFlow includes an algorithm that automatically finds the optimal model placement and parallelism strategies for a given RLHF dataflow and hardware cluster. It explores the design space to find a configuration that minimizes the end-to-end iteration time, tailoring the execution plan to the specific workload and available hardware.

Limitations

While innovative, the framework has several limitations, primarily related to the complexity of its optimization algorithm and its assumptions about the hardware environment.

Mapping Algorithm Complexity

The search for the optimal device mapping is a combinatorial problem and can be computationally expensive, especially for very large clusters. While the framework uses caching techniques to improve efficiency, the process can still take a significant amount of time before the main RLHF job can begin.

Hardware Assumptions

The system is designed and evaluated on homogeneous GPU clusters. Extending its automated mapping capabilities and performance models to support heterogeneous hardware environments, which are common in real-world deployments, remains an area for future work.

Coarse-Grained Resource Sharing

To avoid out-of-memory errors and resource contention, models that are colocated on the same set of GPUs are executed sequentially. This is a safe but potentially inefficient use of resources compared to more fine-grained multiplexing techniques that could allow for concurrent execution.

Future Research Directions

The paper identifies a key area for future work: investigating fine-grained auto-mapping algorithms for GPU sharing in RLHF training, coupled with model offload optimization and integration of heterogeneous devices. This would allow the framework to move beyond simple sequential execution for colocated models, thereby improving overall GPU utilization and system throughput.

Class Discussion

Q: How can HybridFlow determine an optimal device placement of each model to maximize throughput?

A: Hybridflow uses an auto mapping algorithm that explores different model placements and parallelism strategies to identify the optimal configuration that maximizes throughput by minimizing end to end latency.

Q: "What is the dash line for OpenRLHF in the diagrams?"

A: The speaker did not have a clear answer. They mention that the dashed line likely represents that the actor model uses two separate copies of weights, one for training and one for generation.

Q: "The presentation mentioned that existing RLHF systems are inflexible. Why is that the case?"

A: Existing RLHF systems are considered inflexible primarily because they adopt a rigid multi-controller paradigm. This architecture deeply nests the code for computation, collective communication, and point to point data transfers within each worker program. This makes the systems tightly coupled with specific LLM frameworks and difficult to modify. As a result, they're typically hard-coded to support only the PPO algorithm and a single, fixed model placement strategy, which hinders code reuse and makes it challenging to experiment with new RLHF algorithms.

Q: "Is code modification in HybridFlow done at runtime while the post-training is being conducted or at compile time?"

A: The modifications to implement different RLHF algorithms are done at compile time. Users write a single script for the central controller that calls the pre-defined APIs for each model. This script defines the dataflow graph before execution begins.

Q: "If there are few parallel workers and the level of code reuse is low, what is the benefit of this paper's approach?"

A: Even in scenarios with fewer workers, the paper's approach offers significant benefits. A key unique aspect is the 3D-HybridEngine, which manages the different parallelism configurations needed for the actor's training and generation phases. It enables efficient, zero-redundancy resharding between these stages, which is a major bottleneck that other systems handle poorly regardless of scale. The benefit also comes from the hybrid control. The single controller

simplifies development, while the multi-controller paradigm ensures that the computation on the workers is still efficient.

Q: "What is the main goal of the planning ablation (the Auto Device Mapping algorithm)?"

A: The primary goal of the auto-mapping algorithm is to optimize model placement to minimize communication overhead and maximize throughput. It strategically decides how to place models onto the available GPUs. For models that have heavy, frequent communication (like the partitions of a single model during training), it tries to place them on the same node to leverage high-speed interconnects like NVLink. For less frequent, point-to-point communication between different models, it can place them across different nodes.

Q: "Are the parallel techniques used in HybridFlow novel, or is the novelty elsewhere?"

A: The novelty isn't in inventing new fundamental parallelism techniques like tensor or pipeline parallelism, which are well-known. Rather, the innovation lies in how these existing techniques are orchestrated and optimized within a flexible framework. HybridFlow's main contributions are the hybrid programming model that decouples the dataflow from computation, the 3D-HybridEngine for the actor model transition, and the automated mapping algorithm that applies these known parallel techniques in an optimal way. The concept of "innovation" presented here is debatable but it is most certainly clever engineering.

Summary of "AReaL: September 30 2025"

Kai Jun Han (kaijun), Gary Huang (ioi), Tongyuan Miao (tymiao), Annie Jiang (annij)

Problem and Motivation

AReaL addresses the two large system-level inefficiencies in reinforcement learning (RL) for Large Language Models (LLMs) in state-of-art training frameworks: GPU underutilization and poor scalability.

GPU Underutilization During Generation

In synchronous RL systems, the longest sequence in a batch in the generation phase must complete before the training phase can begin, resulting in substantial GPU idling. For RL setups with highly variable generation lengths, this creates a severe bottleneck where faster GPUs idle waiting for stragglers.

Poor Scalability

Synchronous systems distribute generation across all available devices, reducing per-GPU batch size and pushing the system into a memory-bound scenario where adding more devices fails to improve throughput. This fundamentally limits the ability to scale RL systems to larger models and clusters. Models like DeepSeek-R1 and OpenAI's o1 series demonstrate that LLMs with RL achieve breakthrough performance on complex tasks. Without effective and better scaling capabilities, the computational cost becomes prohibitively expensive and inefficient.

Related Works

RL for LLMs

Reinforcement learning has become the predominant approach for training reasoning-capable LLMs, with successful applications in mathematical reasoning, coding, and scientific problem-solving. Open source efforts like DeepScaleR and DeepCoder have demonstrated strong results through distillation from models like DeepSeek-R1.

Synchronous RL Systems

Most existing large-scale systems like veRL, OpenRLHF, and DeepSpeed-Chat operate synchronously by strictly alternating between generation and training. These systems ensure training on the latest model outputs for stability but suffer from underutilization and scalability problems discussed earlier.

Asynchronous RL Systems

With appropriate algorithmic innovations, asynchronous RL has achieved success in game applications. However, asynchronous RL has been largely unexplored in LLM training, except for short-context settings or synchronous training with one/two-step between generation and

training. AReaL extends these prior works and provides a more flexible and adjustable trade-off between staleness and training speed.

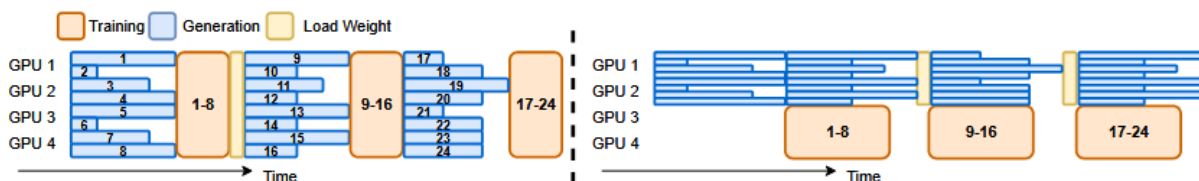


Figure 1: Execution timeline of a synchronous (left) and a one-step overlap (right) RL system showing underutilized inference devices.

Solution Overview

AReaL introduces a fully asynchronous RL system that completely decouples generation from training through an algorithm-system co-design approach.

Core Components

- *Interruptible Rollout Workers*, which continuously generate outputs in a streaming fashion. They can receive weight updates in the middle of a generation, discard old KV caches, and continue generation with new weights.
- *Trainer Workers* that continuously sample from a replay buffer and perform PPO updates whenever the training batch is collected, completely independent and isolated from the generation phase.
- *Rollout Controller* is in charge of coordinating between the rollout workers, reward service, and trainer workers. It does so while maintaining staleness constraints, which are configured via a hyperparameter prior to training that determines the maximum acceptable data scaleness.
- *Decoupled PPO Objective*. Separates behavior policy (for sampling) from proximal policy (for trust region), enabling stable training on stale data.

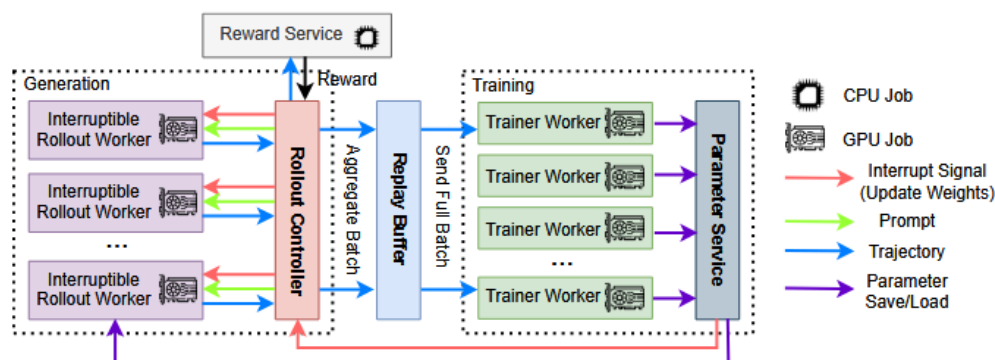


Figure 2: The AREAL architecture featuring asynchronous generation and training components.

Key Insights

Streaming Eliminates Synchronization Bottleneck

The interruptible generation proposed by this paper leads to an overall throughput increase, while dynamic batching yields throughput improvement additionally. Continuous generation without waiting for batch completion increases GPU utilization by 12% to 17%, and achieves up to a 2.77 end-to-end speedup.

Moderate Staleness is Tolerable

Introducing a hyperparameter for the number of policy versions to keep in the replay buffer and used for training allows for controlled data staleness in the whole pipeline. With proper tuning, this can enable substantial performance increase and idleness reduction while having minimal impact on final accuracy. The paper also proposed a few adjustments in the training algorithm to account for and better handle moderately stale data.

Interruptible Trajectories Maintain Correctness

While standard PPO assumes trajectories are sampled from a single policy, interruptible generation produces sequences where different segments come from different policy versions. The paper demonstrates that for any such trajectory, there exists an equivalent behavior policy that could have generated the entire sequence. This equivalence is crucial because it validates the use of the decoupled PPO objective for training.

Limitations

One-Step Overlap Generation Not Compared

The one/two-step overlap generation methods were mentioned in the related works to be the prior state-of-art in LLM with RL training. But this paper did not compare against it in its evaluation and only used the most basic synchronous method without any meaningful attempt to reduce the straggler bottleneck as the baseline.

Manual Staleness Configuration

The system requires manual tuning of the maximum permitted staleness hyperparameter, with different fixed values used for math and coding tasks. The paper does not propose an automated or adaptive method for determining the optimal staleness budget, which is a critical factor in balancing system throughput against potential performance degradation.

Static and Heuristic Resource Allocation

The paper’s architecture relies on a fixed 75-25 split between generation and training devices, which was chosen as a heuristic based on early experiments. This static configuration may not be optimal across different models, hardware setups, or workloads, and the paper does not explore dynamic mechanisms to adjust this resource partition during training to maximize efficiency.

Future Research Directions

Extension to Broader Domains

This paper focuses on mathematical reasoning and code generation, which have clear, rule-based reward functions. Potential future research could apply this asynchronous framework to domains requiring more complex feedback mechanisms. This includes tasks involving learned reward models, direct human feedback, or multi-step agentic workflows where rewards may be sparse or difficult to define, testing the system's flexibility and scalability in new scenarios.

Dynamic Resource Allocation

While this paper decouples generation and training to improve utilization, future work could explore dynamic resource allocation. A more advanced system could monitor the generation throughput and the size of the replay buffer, and dynamically adjust the number of nodes allocated to generation versus training. This would allow the system to adapt to bottlenecks in real-time and further optimize hardware efficiency beyond the current static 75-25 split.

Class Discussion

Q: Decoupled PPO does it still retain the theoretical convergence or is it purely imperial?

A: yes there is a proof that it is correct

Prof Comment:

In the mid 2010s there was a lot of push to move from synchronous to asynchronous training. There were also recent attempts for . (i.e. Meta)

There has to be an assumption that the new one is not too far away than the most recent one, it is going in some predetermined direction and not going entirely at random.

Q: Are there existing methods that allow dynamic resource allocation rather than 75/25 split, which will be difficult because previous papers are about synchronized generation and what we have here is trying to be dynamic and asynchronous generation, also this reinforcement learning

Q: For scalability: it is near linear. What is the staleness set to? If staleness is acceptable to inf, could it scale maximally?

Q: How good was accuracy tuning the scalability? Or if they sacrifice performance for both?

A: The ablation charts fail to specify this relationship. They should have had it clearer.

Q: Interrupting rollouts improves efficiency, does this lead to any bias decreasing accuracy?

A: When we have ablations, we decrease accuracy, therefore, they decoupled PPO objectives.

Taking staleness is fine due to weighting

Does not guarantee will converge to the same as a synchronous pipeline, but it will converge to some local minima (will not diverge)

Q: *With constantly shorter generations, is AReal still beneficial than other RLHF?*

Q: "There are significant idle times ('bubbles') between the actor's generation and training stages. Can we just increase the batch size to make the training stage run longer and reduce the relative impact of these bubbles?"

A: While increasing the batch size to make training longer could amortize the transition cost to some extent, it doesn't solve the core problem of the transition itself being a major bottleneck. Furthermore, simply increasing the global batch size isn't always feasible due to memory constraints or algorithmic requirements.